ML25 A first neural network, and why bigger is not automatically better

Optional extension after session 24. Optional, for interest, no extra class, no extra install.

WHERE THIS IS USED

Many phones recognise speech with neural networks, which are big stacks of small calculations. For a small table, a tree or a line is often enough. Would you reach for the big model first?

This optional notebook is for interest only. It uses scikit-learn's MLPClassifier, a small neural network, on the iris flowers. Inputs are scaled first, because networks learn badly from numbers on different scales. You compare the network with the depth 3 decision tree and with the majority baseline on the same validation flowers.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score
X, y = load_iris(return_X_y=True)
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=7,
    stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
    random_state=7, stratify=y_dev)
scaler = StandardScaler().fit(X_train)
network = MLPClassifier(hidden_layer_sizes=(8,), max_iter=2000,
    random_state=7).fit(scaler.transform(X_train), y_train)
tree = DecisionTreeClassifier(max_depth=3, random_state=7).fit(X_train, y_train)
baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
net_val = accuracy_score(y_val, network.predict(scaler.transform(X_val)))
tree_val = accuracy_score(y_val, tree.predict(X_val))
base_val = accuracy_score(y_val, baseline.predict(X_val))
print(net_val, tree_val, base_val)
```

On 150 flowers a neural network has no advantage over a small tree. A bigger model is not automatically a better one, and it needs more data, more care and more checking.